

Layouts, Templates & Nested Routing

Beginner → Intermediate → Expert | Interview Q&A with Answers

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

What is a Layout?

A layout is **shared UI that wraps multiple pages** and does NOT re-render when you navigate between pages inside it. It is the persistent shell of your app — think Navbar, Sidebar, Footer. Only the inner content swaps on navigation.

■ Key Point: Navigating from `/dashboard` to `/dashboard/settings` — the Navbar and Sidebar stay mounted. Only the page content in `{children}` swaps.

```
// app/layout.tsx — Root Layout (REQUIRED — wraps entire app)
// Only the ROOT layout has <html> and <body> tags
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <Navbar />           {/* Shows on EVERY page */}
        <main>{children}</main> /* Page content swaps here */}
        <Footer />          {/* Shows on EVERY page */}
      </body>
    </html>
  );
}
```

What is a Template?

A template uses the **same syntax as a layout** but behaves differently: it creates a **fresh new instance** on every navigation. State is NOT preserved — effects re-run, counters reset, forms clear.

```
// app/template.tsx — same syntax as layout.tsx
export default function Template({ children }: { children: React.ReactNode }) {
  return <div>{children}</div>;
}
```

Layout vs Template — Side-by-Side Comparison

Feature	layout.tsx	template.tsx
---------	------------	--------------

Re-renders navigation?	on	No — persists, same instance	Yes — fresh instance each time
State preserved?		Yes — useState stays intact	No — state resets on navigate
useEffect re-runs?		No — only on mount	Yes — re-runs on every navigate
When to use?		Navbar, sidebar, auth wrapper	Entry animations, per-page analytics
File name		layout.tsx	template.tsx

Special Files Quick Reference

File	Purpose	Key Behaviour
layout.tsx	Shared UI shell for a route segment	Persists across nav, no re-render
template.tsx	Like layout but fresh per navigation	New instance, state resets
page.tsx	The actual page UI — makes route public	Re-renders on every visit
loading.tsx	Skeleton/spinner while page loads	Auto Suspense boundary
error.tsx	UI shown when component throws	Must be "use client"
not-found.tsx	Custom 404 for segment	Shown on notFound() call

■ INTERMEDIATE LEVEL

Nested Layouts — How They Stack

Every layout wraps ALL the layouts and pages below it in the folder hierarchy. This creates a nested shell structure automatically — no manual configuration needed.

```
app/
  layout.tsx      ← Level 1: Root layout (wraps everything)
  page.tsx        → /
  dashboard/
    layout.tsx    ← Level 2: Dashboard layout
    page.tsx      → /dashboard
    settings/
      layout.tsx  ← Level 3: Settings layout
      page.tsx    → /dashboard/settings
```

What Next.js renders for `/dashboard/settings`:

```
// All THREE layouts wrap each other automatically:
<RootLayout>
  <DashboardLayout>
    <SettingsLayout>
      <SettingsPage />
    </SettingsLayout>
  </DashboardLayout>
</RootLayout>
```

Real Code — Three-Level Layout Example:

```
// app/layout.tsx – Root (MUST have html + body)
export default function RootLayout({ children }) {
  return (
    <html lang="en"><body>
      <Navbar />
      {children}
    </body></html>
  );
}

// app/dashboard/layout.tsx – Dashboard shell with sidebar
export default function DashboardLayout({ children }) {
  return (
    <div className="flex">
      <Sidebar />           {/* Only on /dashboard/* */}
      <div className="flex-1">{children}</div>
    </div>
  );
}

// app/dashboard/settings/layout.tsx – Settings with tabs
export default function SettingsLayout({ children }) {
  return (
    <div>
      <SettingsTabs />       {/* Only on /dashboard/settings/* */}
      {children}
    </div>
  );
}
```

Partial Rendering — Why Layouts Are Fast

When navigating between routes that share a layout, Next.js **only re-renders the segments that changed**. Layouts above the changed segment stay mounted — no re-render, no re-fetch, React state is preserved.

```
// Navigate: /dashboard/analytics → /dashboard/settings

BEFORE:  RootLayout + DashboardLayout + AnalyticsPage
AFTER:   RootLayout + DashboardLayout + SettingsPage
         ↑
         These stay mounted – zero re-render!
         Only SettingsPage unmounts and remounts.

// Real benefit: Sidebar data NOT re-fetched on navigation
// Music player in layout keeps playing across page changes
// Form state in layout is preserved
```

■ Partial Rendering is why Next.js apps feel instant on navigation. The layout is already in memory — React just swaps the page component.

Metadata in Layouts — SEO Made Easy

Export a **metadata** object or **generateMetadata()** function from layout.tsx or page.tsx. Next.js automatically builds the HTML head tags. Metadata from layouts is inherited and overridden by pages.

```
// app/layout.tsx — Base metadata (inherited by all pages)
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: {
    template: '%s | Syngenta Portal', // %s = replaced by page title
    default: 'Syngenta Portal',      // Fallback if page has no title
  },
  description: 'Syngenta India Agri Platform',
};

// app/products/page.tsx — Page-level metadata
export const metadata: Metadata = {
  title: 'Products', // Final: 'Products | Syngenta Portal'
  description: 'Browse our full product catalog',
};

// app/products/[id]/page.tsx — Dynamic metadata
export async function generateMetadata({
  params,
}: {
  params: { id: string };
}): Promise<Metadata> {
  const product = await fetchProduct(params.id);
  return {
    title: product.name,          // "Wheat Hybrid 50 | Syngenta Portal"
    description: product.summary,
    openGraph: { images: [product.imageUrl] },
  };
}
```

loading.tsx — Automatic Suspense Boundary

Simply create a **loading.tsx** file and Next.js automatically wraps your page in a React Suspense boundary. The layout stays interactive while the page streams in. You get streaming UI for free — no manual Suspense setup needed.

```
// app/dashboard/loading.tsx
// Shown INSTANTLY while dashboard data fetches
// Layout (Navbar/Sidebar) stays fully interactive
export default function DashboardLoading() {
  return (
    <div className="animate-pulse space-y-4">
      <div className="h-8 bg-gray-200 rounded w-48" />
      <div className="h-4 bg-gray-200 rounded w-full" />
      <div className="h-4 bg-gray-200 rounded w-3/4" />
    </div>
  );
}

// What Next.js does internally (automatically):
<Suspense fallback=<{DashboardLoading /}>>
  <DashboardPage />    {/* Streams in when server is ready */}
</Suspense>
```

error.tsx — Automatic Error Boundary

error.tsx catches runtime errors in its route segment and **MUST** be a Client Component. It receives the error object and a `reset()` function to retry rendering.

```
// app/dashboard/error.tsx
'use client'; // REQUIRED – error boundaries need Client Component
import { useEffect } from 'react';

export default function DashboardError({
  error,
  reset,    // Function to retry rendering this segment
}): {
  error: Error & { digest?: string };
  reset: () => void;
} {
  useEffect(() => {
    console.error("Dashboard error:", error); // Log to Sentry etc.
  }, [error]);

  return (
    <div>
      <h2>Something went wrong!</h2>
      <button onClick={reset}>Try Again</button>
    </div>
  );
}
```

■ ■ Important: **error.tsx** does NOT catch errors in **layout.tsx** of the same segment — because **error.tsx** lives inside the layout. To catch layout errors, place **error.tsx** in the PARENT segment.

■ EXPERT LEVEL

How Layout Persistence Works Internally

React renders a component tree as a tree of fibers. When Next.js navigation occurs, it compares the old and new route trees. Segments that share the same layout component at the same position in the tree are kept as-is — React does not unmount and remount them. Only the changed leaf segment gets unmounted and a new page

component mounts in its place.

```
// React fiber tree – before navigation
RootLayout (fiber alive)
  DashboardLayout (fiber alive)
    AnalyticsPage (fiber alive)

// After navigating to /dashboard/settings:
RootLayout (same fiber – state preserved)
  DashboardLayout (same fiber – state preserved)
    SettingsPage (NEW fiber – fresh mount)

// Result:
// → Music player in DashboardLayout keeps playing
// → Sidebar scroll position preserved
// → Only SettingsPage mounts fresh
```

Passing Data from Layout to Page — Correct Patterns

Layouts receive ONLY the children prop. They cannot directly pass props to page components. Use one of the three correct patterns below:

```
// Pattern 1: React Context (Client Components only)
'use client';
import { createContext, useContext } from "react";
const UserCtx = createContext(null);

export function UserProvider({ children, user }) {
  return <UserCtx.Provider value={user}>{children}</UserCtx.Provider>;
}
export const useUser = () => useContext(UserCtx);

// In layout.tsx (wrap children with provider):
export default async function DashboardLayout({ children }) {
  const user = await getUser();
  return <UserProvider user={user}>{children}</UserProvider>;
}

// Pattern 2: Fetch independently – Next.js deduplicates automatically
// Calling fetch() with same URL in layout AND page = one network request
export default async function DashboardLayout({ children }) {
  const user = await fetch('/api/user').then(r => r.json()); // Request 1
  return <div><UserBadge user={user} />{children}</div>;
}
export default async function DashboardPage() {
  const user = await fetch('/api/user').then(r => r.json()); // Deduplicated!
  return <h1>Welcome {user.name}</h1>;
}

// Pattern 3: cookies/headers (Server Components – auth)
import { cookies } from 'next/headers';
export default async function DashboardLayout({ children }) {
  const cookieStore = cookies();
  const token = cookieStore.get("auth-token");
  if (!token) redirect("/login");
  return <>{children}</>;
}
```

Root Layout Rules — Critical Knowledge

```
// ROOT layout (app/layout.tsx) — MUST have html and body tags
export default function RootLayout({ children }) {
  return (
    <html lang="en">      {/* Only here in the entire app */}
      <body>{children}</body>
    </html>
  );
}

// NESTED layouts — NO html or body
export default function DashboardLayout({ children }) {
  return (
    <section className="dashboard">
      <Sidebar />
      {children}  {/* Correct — no html/body */}
    </section>
  );
}

// WRONG — html/body in nested layout = React hydration error
export default function BadLayout({ children }) {
  return (
    <html>      {/* ERROR: nested html tag */}
      <body>{children}</body>
    </html>
  );
}
```

Colocation — Keeping Files Close to Routes

Only **page.tsx** makes a route publicly accessible. Other files inside `app/` (components, utils, tests) are colocated but private — they are never exposed as URL routes.

```
app/
├── dashboard/
│   ├── page.tsx      → /dashboard   (PUBLIC — accessible via URL)
│   ├── layout.tsx    → Used internally (NOT a route)
│   ├── loading.tsx   → Used internally (NOT a route)
│   └── components/
│       ├── Chart.tsx → NOT accessible via URL (colocated)
│       ├── Stats.tsx → NOT accessible via URL (colocated)
│       └── utils/
│           └── helpers.ts → NOT accessible via URL (colocated)
```

Critical Gotchas

■ Gotcha 1: Root layout is **MANDATORY**. `app/layout.tsx` must always exist and must contain html and body tags. Deleting it causes a build error.

■ Gotcha 2: `error.tsx` **MUST** have "use client" at the top. React Error Boundaries are class-based and only work in Client Components. Forgetting this causes a build error.

■ Gotcha 3: `error.tsx` does **NOT** catch errors thrown by `layout.tsx` in the same folder. It lives inside the layout. To catch layout errors, add `error.tsx` to the parent folder.

■ Gotcha 4: Metadata export (const metadata or generateMetadata) only works in Server Components. Putting it in a "use client" component silently ignores it — no error, just missing SEO tags.

■ Best Practice: Use loading.tsx for every data-fetching route segment. It gives you streaming + skeleton UI automatically without any manual Suspense setup.

■ Best Practice: Keep layout.tsx as a Server Component. If you need interactivity in the layout (like a hamburger menu), extract just that part into a separate Client Component.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is a layout in Next.js App Router?

Answer: A layout is shared UI that wraps multiple pages within a route segment and persists across navigation without re-rendering. It is defined in layout.tsx and receives a children prop. Layouts are ideal for Navbar, Sidebar, and Footer — elements that should stay mounted as the user navigates between pages.

BEGINNER

Q2. What is the difference between layout.tsx and template.tsx?

Answer: Both wrap child pages with the same syntax. layout.tsx reuses the same mounted component instance across navigations — state is preserved and effects do not re-run. template.tsx creates a fresh new instance on every navigation — state resets and useEffect re-runs. Use template for entry animations, per-page analytics events, or UI that must reset on each page visit.

BEGINNER

Q3. Does the Root layout need html and body tags? What about nested layouts?

Answer: Yes — app/layout.tsx (root layout) MUST contain html and body tags. It is the only layout that should have them. Nested layouts (e.g. app/dashboard/layout.tsx) must NOT have html or body tags — adding them causes a React hydration error because you would be nesting html inside html.

Intermediate Level

INTERMEDIATE

Q4. How does nested layout work? Explain with /dashboard/settings.

Answer: Every folder in app/ can have its own layout.tsx. Next.js automatically stacks layouts from outside in. For /dashboard/settings: RootLayout wraps DashboardLayout which wraps SettingsLayout which wraps SettingsPage. On navigation between dashboard routes, only the innermost page component swaps. Outer layouts stay mounted — this is called Partial Rendering.

INTERMEDIATE**Q5. What is Partial Rendering and why does it matter for performance?**

Answer: Partial Rendering means Next.js only re-renders route segments that changed on navigation. Layouts shared between the old and new route stay mounted in the React tree — no re-render, no re-fetch. This preserves state (sidebar scroll, music player), avoids redundant API calls, and makes navigation feel instant.

INTERMEDIATE**Q6. How do you handle loading states in Next.js without manually writing Suspense?**

Answer: Create a `loading.tsx` file in the route folder. Next.js automatically wraps the page in a Suspense boundary with `loading.tsx` as the fallback. The layout above stays interactive while the page streams in. You get skeleton UI and streaming for free — no manual Suspense, no `useEffect` loading state needed.

INTERMEDIATE**Q7. Why must `error.tsx` be a Client Component?**

Answer: `error.tsx` uses React Error Boundary under the hood. Error Boundaries are implemented as class components and require lifecycle methods only available in Client Components. The `reset()` prop also requires client-side React to trigger a re-render. Forgetting "use client" causes a build error. Additionally, `error.tsx` does not catch errors thrown by its sibling `layout.tsx` — put it in the parent to catch layout errors.

INTERMEDIATE**Q8. How do you add SEO titles to pages with a shared suffix like "| Syngenta"?**

Answer: In the root `layout.tsx`, export metadata with `title.template` set to `"%s | Syngenta Portal"` and a default fallback title. Each `page.tsx` exports its own metadata with a title string — Next.js fills `%s` with that string automatically. For dynamic pages, export an `async generateMetadata()` function that receives `params` and returns a Metadata object.

Expert / Syngenta-Style Questions

EXPERT**Q9. How does Next.js preserve layout state across navigation internally?**

Answer: Next.js uses React's reconciliation algorithm. Layouts at the same position in the component tree keep the same React fiber across navigations. Only the leaf page component changes fibers (unmounts old, mounts new). This means `useState`, `useRef`, scroll position, and any running timers in layouts survive navigation completely untouched.

EXPERT**Q10. A layout needs to share user data with all child pages. What are the correct patterns?**

Answer: Three patterns: (1) React Context — create a context in the layout, wrap children with `Provider`; pages consume with `useContext`. (2) Independent fetching with deduplication — fetch the same URL in both layout and page; Next.js deduplicates identical `fetch()` calls in one render pass = one network request. (3) `cookies()/headers()` for auth data — read server-side cookies in layout, redirect if not authenticated.

SYNGENTA**Q11. Design the layout architecture for a Syngenta portal with public and authenticated sections.**

Answer: app/layout.tsx: Root layout with html/body, global styles, base metadata template.
app/(marketing)/layout.tsx: Full-width public layout, no sidebar, public metadata.
app/(portal)/layout.tsx: Auth check via cookies(), redirect to /login if missing token, renders portal shell with sidebar and nav. app/(portal)/dashboard/layout.tsx: Dashboard-specific layout with crop filter panel and data date selector. Route groups keep URLs clean. Each layout layer adds only what it owns.

EXPERT**Q12. What is the difference between loading.tsx and a manual Suspense boundary?**

Answer: loading.tsx is a file convention — Next.js automatically wraps the entire page.tsx in a Suspense boundary with loading.tsx as the fallback. It is segment-level and applies to the whole page. Manual Suspense gives finer control — you can wrap individual components within a page, allowing different parts to stream in independently with different fallbacks. Use loading.tsx for page-level loading. Use manual Suspense for component-level streaming (e.g. a slow chart while the rest of the page renders).